

What should I name
my new programming
Language?

Jule → you-la

↑ my name

CS 340

clicker.cs.illinois.edu

Q1

~Code~
340



A Program's Memory | Virtual Memory

Justin P.



Sage Y.



mp 2 secrets!

"Arr me see green
turtle"

Updates

1. MP3 - Due Today



bytes → each bit indicates
if something is
in the set

1. MP4 - Out today, due next week

a. No MPP this week

char 1 - - - -
↑
orange

1. HW5 - due Wednesday

- retake exam

→ Thursday

read word from file

turn into int set-bitset

store in array

⋮

loop

loop

loop

Program's Memory | Virtual Memory

Today's LGs - Deepen your understanding of how memory works in a C program and on your computer.

Be able to differentiate between globals/static, the stack, and the heap

Be able to identify common issues with the stack and the heap use

Be able to implement a simplified, efficient version of the C allocator

Be able to articulate why we use virtual memory in a computer

Be able to put together system concepts with virtual memory.

↑ MP4

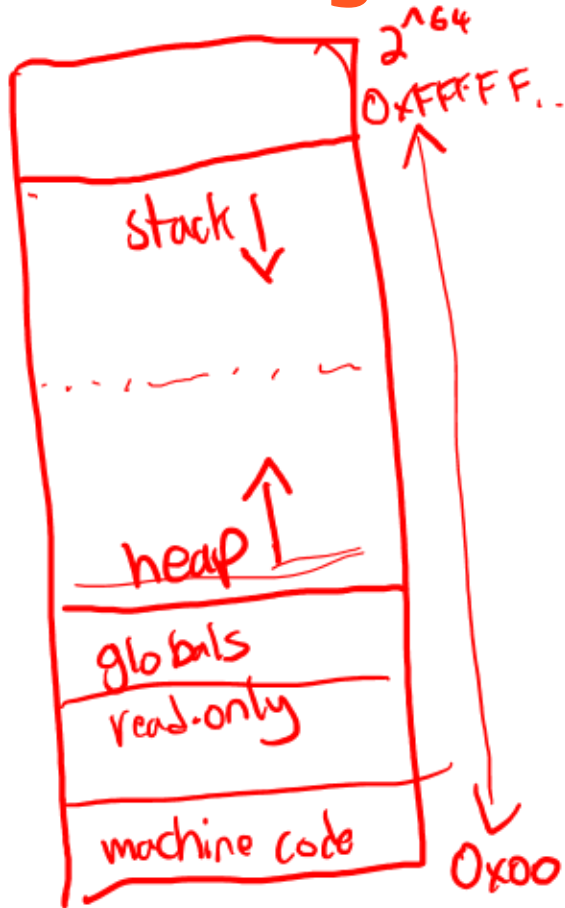
Agenda

1. A Program's Memory
 - a. Description
 - b. Examples
 - c. Clickers

2 MP4 intro

- 3 Virtual Memory
 - a. Description
 - b. Examples
 - c. Clickers

The C Program's View of its Memory Space



The Stack

- stack frame for each function call
 - local variables, extra
- = Stack has a fixed size

errors

- stack overflow - too many stack frames active
- use of memory after function returns

The Heap

- used in C with malloc, etc..
- program requests use of ~~those~~^{that} memory

errors

- Bad free
- memory leak - can lead to running out of space

Code/Globals/Static Memory

Code - machine code 1's and 0's
can't edit

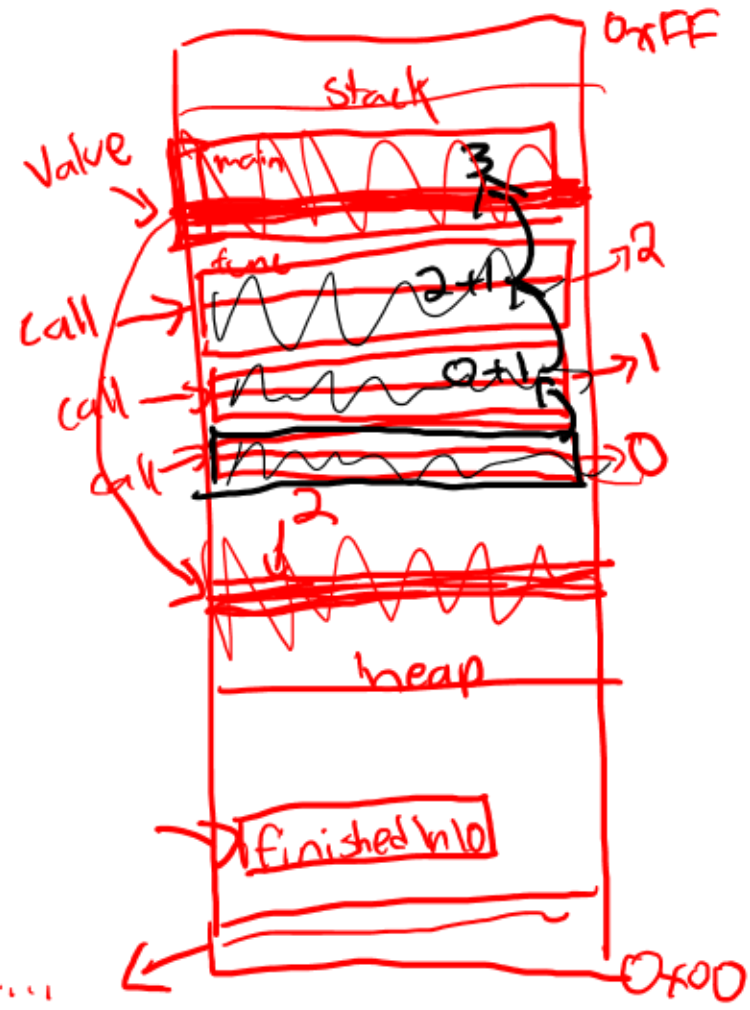
Read only globals - string literals
`char * str = "hello";`

Static and global variables

↓
not tied
to a
function

Example

```
4  int func(int call){
5      → if(call == 0) return 0;
6      → return func(call-1) + call;
7  }
8
9  → int main(){
10     int* value = malloc(sizeof(int));
11     *value = 2;
12     → if(func(*value) > 2){
13         printf("finished\n");
14     }
15     free(value);
16 }
```



Is there an error?

no base-case

```
4  int func(int call){
5      return func(call-1) + call;
6  }
7
8  int main(){
9      int* value = malloc(sizeof(int));
10     → *value = 4;
11     if(func(*value) > 7){
12         printf("finished\n");
13     }
14     free(value);
15 }
```

clicker.cs.illinois.edu

Q2

~Code~
340



- A) No
- B) Compiler error
- C) Runtime - memory leak
- D) Runtime - stack overflow**

Is there an error?

```
6  int *func(int y){
7      int x = y + y + y;
8      int *ptr = &x;
9      return ptr;
10 }
11
12 int main(){
13     int *ptr = func(7);
14     printf("%d\n", *ptr);
15 }
```

PROBLEMS OUTPUT **TERMINAL 3**

TERMINAL

```
root@7a6ba03ac098: /workspaces/bst#
```



clicker.cs.illinois.edu

Q3

~Code~
340



- ☒ A) Yes - undefined behavior
- ~~B) No - looks good~~
- ~~C) Yes - memory leak~~
- ~~D) Yes - stack overflow~~

heap →

stack →

21

Explanation

```
6  int *func(int y){
7      int x = y + y + y;
8      int *ptr = &x;
9      return ptr;
10 }
11
12 int main(){
13     int *ptr = func(7);
14     printf("%d\n", *ptr);
15 }
```

PROBLEMS

OUTPUT

TERMINAL 3

✓ TERMINAL

```
root@7a6ba03ac098:/workspaces/bst#
21
```

```
6  int *func(int y){
7      int x = y + y + y;
8      int *ptr = &x;
9      return ptr;
10 }
11
12 int main(){
13     int *ptr = func(7);
14     func(1);
15     printf("%d\n", *ptr);
16 }
17
```

PROBLEMS

OUTPUT

TERMINAL 3

✓ TERMINAL

● root@7a6ba03ac098: /workspaces/bst#

3



What is true about this code?

```
6  int main(){
7      char *str = "hello";
8      char str2[6] = "hello";
9
10     printf("%#X and %#X\n", str, str2);
11 }
```



clicker.cs.illinois.edu

Q4

~Code~
340



- ☒ (A) str and str2 will always hold different addresses
- ☒ (B) str will always hold a smaller address than str2
- ☒ (C) str lives on the heap but holds the address to read only memory

str != str2



How many bytes do str + str2 need on the stack? Assuming 8 byte addresses.

```
6  int main(){
7  8 bytes [char *str = "hello";
8  6 bytes [char str2[6] = "hello";
9
10     printf("%#X and %#X\n", str, str2);
11 }
```

clicker.cs.illinois.edu

Q5

~Code~
340



- A) 12 bytes
- B) 14 bytes**
- C) 16 bytes
- D) 18 bytes

Agenda

1. A Program's Memory
 - a. Description
 - b. Examples
 - c. Clickers

1. MP4 intro

1. Virtual Memory
 - a. Description
 - b. Examples
 - c. Clickers

MP4 - Allocator

Premise you are in charge of a big chunk of memory
- other programs/code will ask for x bytes of that
memory

MP4 - Allocator

- Goal - Keep this functionality but improve space and time

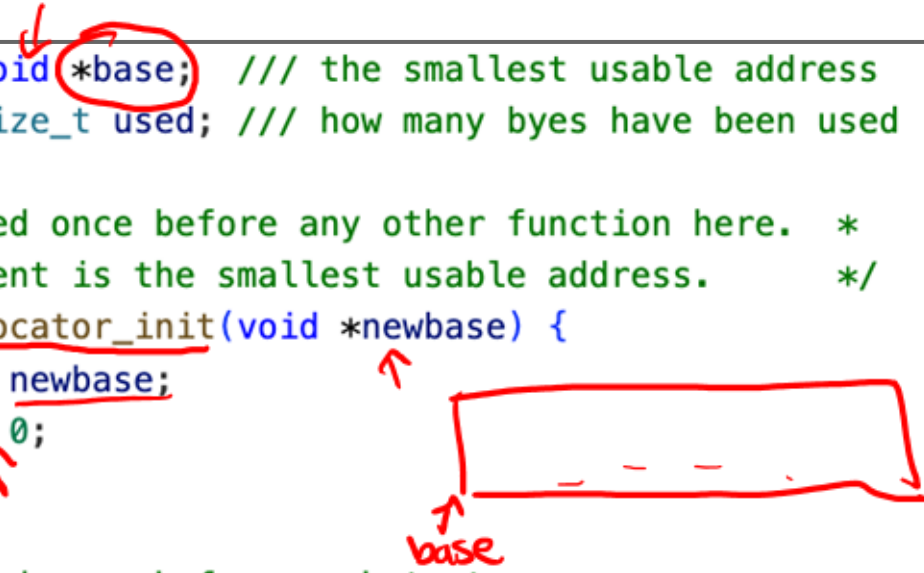
`malloc(size)` - allocates size bytes and returns a pointer to the allocated memory.

`free(p)` - frees the memory space pointed to by p which must have been allocated previously with a call to `malloc`.

`realloc(p, size)` - changes the size of allocated memory pointed to by p to size bytes. Contents of memory will be unchanged if within new range. If size is 0, this is the same as `free(p)`. The function returns a pointer to the new block of allocated memory or NULL if size is 0.

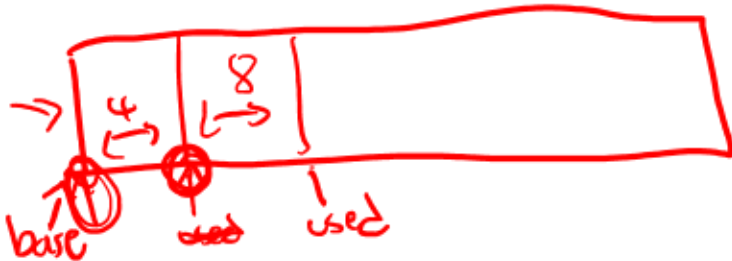
MP4 - Starter Code

```
20  static void *base;  /// the smallest usable address
21  static size_t used;  /// how many bytes have been used
22
23  /** Called once before any other function here.  *
24   * Argument is the smallest usable address.      */
25  void allocator_init(void *newbase) {
26      base = newbase;
27      used = 0;
28  }
29
30  /** Called once before each test case.  *
31   * Free any used memory and reset for the next test. */
32  void allocator_reset() { used = 0; }
```



MP4 - Starter Code

```
35  /** Like malloc but using the memory provided to allocator_init */
36  void *mymalloc(size_t size) {
37      // simple version pushes each block on a stack
38      → void *ans = base + used;
39      used += size;
40      return ans;
41  }
42
43  /** Like free but using the memory provided to allocator_init */
44  → void myfree(void *ptr) {
45      // simple version does nothing
46  }
```



MP4 - Starter Code

```
48  /** Like realloc but using the memory provided to allocator_init */
49  void *myrealloc(void *ptr, size_t size) {
50      // simple version does malloc-copy-free
51      if (!size) { myfree(ptr); return NULL; }
52      void *ans = mymalloc(size);
53      if (ptr && ans) {
54          memcpy(ans, ptr, size); // should be min(size, oldsize) but oldsize unknown
55          myfree(ptr);
56      }
57      return ans;
58  }
```

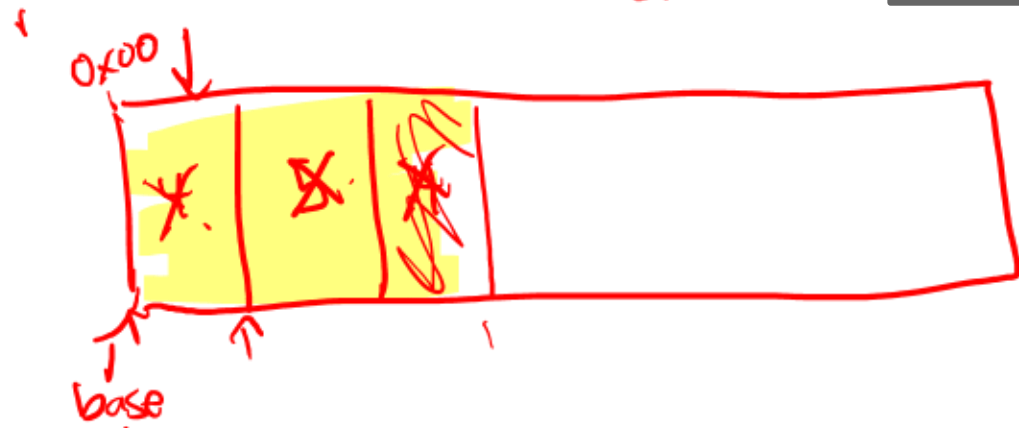


MP4 - Example

used = ~~20~~ ~~13~~ 13

How less space? ↑

uses
13 bytes
of
space



```
3  const char *mytest(allocator *a) {  
4  → void *ptr = a->malloc(4);  
5  → void *ptr2 = a->malloc(5);  
6  → a->free(ptr);  
7  → a->free(ptr2);  
8  → void *ptr3 = a->malloc(4);  
9  if(ptr3 < ptr2){  
10     return "not the base algorithm";  
11 }  
12 → return 0;  
13 }
```

Steps 1-6

Steps 7 and 8 - 0.5% total
extra credit

Agenda

1. A Program's Memory

- a. Description
- b. Examples
- c. Clickers

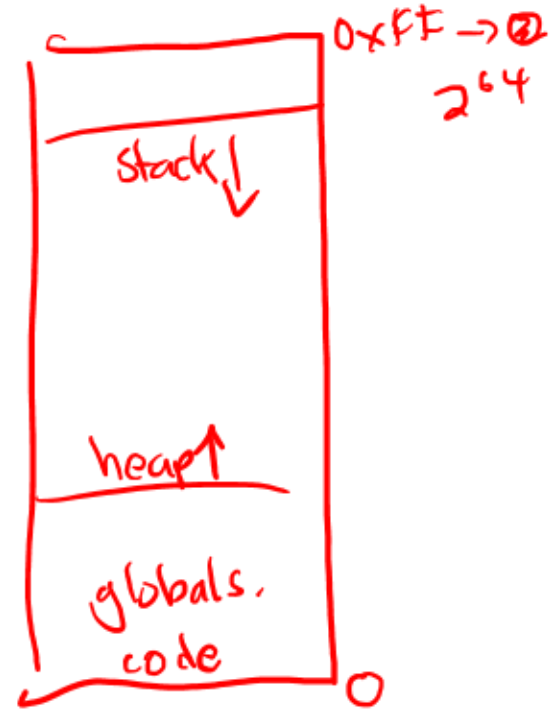
1. MP4 intro

1. Virtual Memory

- a. Description
- b. Examples
- c. Clickers

Program Memory - Takeaways

- A C program sees its addresses as starting at 0 and going until 2^{64} .
- C code ->
- Heap
- Stack



What is the following translated to?

2^{64} Bytes $\rightarrow 2^{60} \cdot 2^4$

Value	base-10	Prefix	Pronounced
2^{10}	1024	Ki	Kilo
2^{20}	1,048,576	Mi	Mega
2^{30}	1,073,741,824	Gi	Giga
2^{40}	1,099,511,627,776	Ti	<u>Tera</u>
2^{50}	1,125,899,906,842,624	Pi	Peta
2^{60}	1,152,921,504,606,846,976	Ei	Exa

clicker.cs.illinois.edu

Q6

~Code~
340

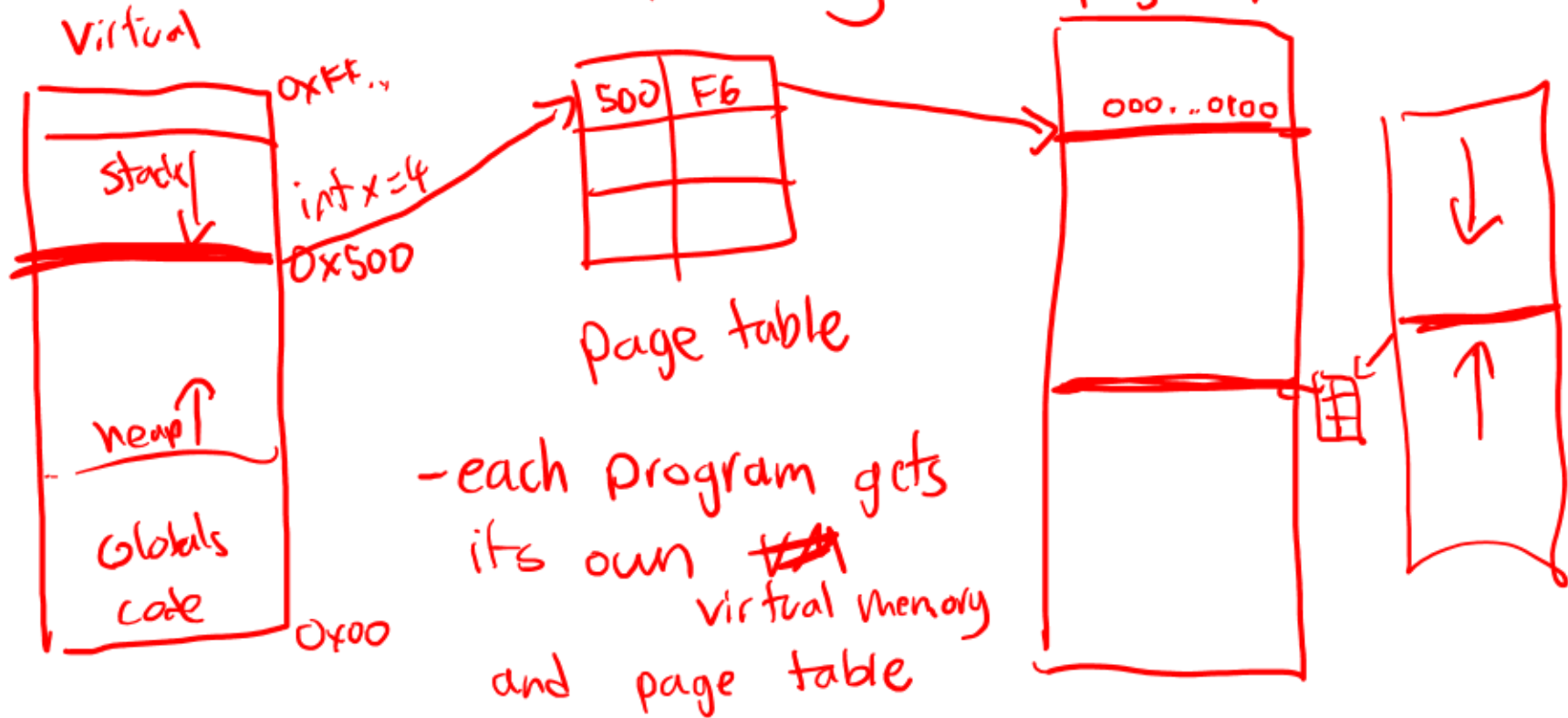


- A) 16 TB
- B) 32 GB
- C) 16 PB
- D) 32 EB
- E) 16 EB

Potential Issues

- How does every program have addresses $0-2^{64}$
- Space — 3 running programs — how does your RAM have that space?
- Security — can I access memory in a different program

The Solution - Virtual memory



- each program gets
its own ~~virtual~~
virtual memory
and page table

- in the CPU - the MMU
handles all the translations

Virtual Memory

- security - separates physical address from the programs

- space

page - 4kB

page table maps

Virtual pages to physical pages

Virtual Memory Example #1

```
char x = 5;  
printf("%#x", &x);  
//Lots of other code  
x = x >> 3;
```

0xFES



0xFES



Physical



① load 0xFES into reg1

② Shift reg1 by 3 and store in reg1

1. CPU decodes ① → MMU

2. MMU use the page table to find corresponding physical address

3. search through the cache for address 0x70

4. retrieve those bytes and surrounding into higher cache

5. store that byte (0x05) in reg1

6. CPU decode ② → ALU

7. move reg1 into ALU with #3 - store result in reg1

fast
small



RAM

0x70

Virtual Memory Example #2

```
int x = -50;  
printf("%#x", &x);  
//Lots of other code  
x = x + 1;
```

50 = 0000 0000 0000 0000 0000 0000 0011 0010

flip = 1111 1111 1111 1111 1111 1111 1100 1101 ← -50

add1 = 1111 1111 1111 1111 1111 1111 1100 1110

0x FFFFFFFE

0x CE, 0xFF, 0xFF, 0xFF ← little endian

① load 0x B50 reg1

② add 1 reg1 store in reg1

③ CPU fetches and decodes ①

2. employ the MMU to translate 0xB50 to its physical address using the page table

3. search cache and memory for 0x80 →

FE	FF	FF	FF
----	----	----	----

4. with the load cpu grabs 4-bytes into reg1 as 0xFFFFFFFFCE

81 82 83 84
0x80

5. next instruction

6. use the ALU to add 1 → reg1 = 0xFFFFFFFFCF = -49

Takeaways

- Virtual memory - isolates programs for security and the illusion of space.
- You (the programmer) cannot see or know what physical address your code is using. You can only access the virtual addresses for your specific program.
- Each running program gets its own page table. Virtual addresses may overlap between programs but physical addresses cannot.
- Physical memory for a program can vary in size and location.
- Virtual memory has a fixed size and a consistent location.

Takeaways II

- The MMU reads the page table to translate addresses from virtual to physical.
- Physical memory with physical addresses use the cache for speed.
- Each page has permission bits that are checked at runtime.

Program's Memory | Virtual Memory

Today's LGs - Deepen your understanding of how memory works in a C program and on your computer.

Be able to differentiate between globals/static, the stack, and the heap

Be able to identify common issues with the stack and the heap use

Be able to implement a simplified, efficient version of the C allocator

Be able to articulate why we use virtual memory in a computer

Be able to put together system concepts with virtual memory.